

GIU Berlin: IET Networks

Bachelor Thesis

MANTA: Measuring the Detection-Privacy Trade-off in Metadata-Only Mobile Traffic Monitoring

A public-corpus study of encrypted-traffic anomaly detection,
telemetry reduction, and observer leakage

Author:	Mahmoud Elfeel
Supervisor:	Prof. Mohamed Ashour
Industry Partner:	Resistine GmbH
Submission Date:	July 2026

Abstract

Mobile and endpoint devices generate continuous traffic across Wi-Fi, cellular, enterprise, and home networks. Defenders need evidence from this traffic to detect compromise, malware-like behavior, beaconing, data-exfiltration patterns, and unusual destination changes. Modern traffic is largely encrypted, so payload inspection is limited and privacy-invasive. Metadata can support threat detection, but it can also reveal application identity, destination behavior, and user activity.

This thesis implements and evaluates MANTA (Mobile Anomaly and Network Threat Analysis), a metadata-only framework for encrypted mobile and endpoint traffic monitoring. The objective is to measure whether Android-computable flow metadata can preserve useful anomaly-detection signal while reducing the privacy leakage of released telemetry. Here, an anomaly is suspicious network behavior inferred from flow-level evidence such as timing, direction, volume, novelty, destination change, burst structure, and per-app baseline deviation. MANTA does not inspect payloads. It converts packet and flow observations into feature windows for local scoring, privacy-controlled export, and reproducible machine-learning evaluation.

The implementation combines an Android endpoint, a backend adapter, and an ML pipeline. The evaluation uses a public-only corpus of 6,034,487 flows from seven dataset sources and 904,371 aggregated 60-second windows. It compares local, backend-assisted, privacy-aware, federated, and privacy-reduced variants under grouped, source-aware evaluation.

The results show that metadata-only detection is feasible, but strongly dependent on feature and deployment alignment. In the shared Android adaptive-window comparison, High-recall RF reaches F1 0.925, Privacy RF reaches F1 0.910, Adaptive hybrid RF reaches F1 0.895, the calibrated backend model reaches F1 0.888, and Low-FPR RF reaches F1 0.817 with fewer false positives.

The privacy evaluation focuses on two leakage risks. For release leakage, the exported feature representation is tested. The `medium` and `strict` views reduce exact app re-identification from 0.282 normalized app-ID leakage in the full view to 0.151 and 0.135, but broader semantic leakage remains high. For observer leakage, a passive on-path observer sees the encrypted traffic sequence before MANTA applies export-time reductions. That benchmark shows severe leakage: normalized leakage is 0.544 for app ID and 0.981 for

dataset source. MANTA improves released-telemetry control, not transport-level privacy.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 Terminology and Security Framing	1
1.3 Problem Statement	2
1.4 Research Questions	3
1.5 Contributions	4
1.6 Thesis Structure	4
2 Background and Related Work	5
2.1 Android Networking and VpnService	5
2.2 Flow-Based Monitoring and Metadata-Only Detection	5
2.3 Encrypted Traffic Analysis and Observer Leakage	6
2.4 Anomaly Detection for Network Security	6
2.5 Privacy-Preserving Traffic Representation	7
2.6 Privacy-Aware Representation Learning and Collaboration	7
2.7 Positioning	7
3 System Architecture and Design	8
3.1 System Overview	8
3.2 Data Flow	9
3.3 Threat Model and Assumptions	9
3.4 Privacy Model	10
3.5 Telemetry-Reduction Design Decisions	10
3.6 Deployment and Evaluation Boundaries	11

4	Implementation	12
4.1	Android Endpoint Implementation	12
4.1.1	VPN Capture and Userspace Forwarding	12
4.1.2	Feature Windows and On-Device Detection	12
4.1.3	UI and Triage Workflow	14
4.2	Backend Adapter Implementation	14
4.3	ML Pipeline Implementation	15
4.3.1	Privacy Views	15
4.3.2	Privacy Student and Federated Path	15
4.3.3	Reproducibility Support	16
5	Experimental Methodology and Evaluation	17
5.1	Experimental Setup	17
5.2	Datasets and Trace Collection	17
5.3	Metrics	18
5.4	Android-Deployable Experiment Methodology	18
5.5	Anomaly-Detection Results	19
5.6	Experiment Progression and Negative Results	20
5.7	Android-Deployable Model Development	21
5.7.1	Shared-Protocol App Model Comparison	22
5.8	Release-Privacy Utility Trade-off	22
5.8.1	Hashing Versus Deleting Reduced Features	23
5.9	Metadata Leakage from Released Views	23
5.10	Local and Remote Models Under Privacy Views	24
5.11	Observer Leakage from Encrypted Traffic	24
5.12	Privacy Student, Federated Path, and Performance	25
5.13	Interpretation	25
6	Privacy, Ethics, and Limitations	27
6.1	Privacy Impact	27
6.2	Ethical Considerations	28
6.3	Release Privacy versus Observer Leakage	28
6.4	Interpretation of the Negative Result	28

6.5	Leakage and Utility Evaluation	29
6.6	Limitations and Threats to Validity	29
7	Conclusion and Future Work	31
7.1	Conclusion	31
7.2	Future Work	32
A	Reproducibility Package	33
A.1	Report Bundle	33
A.2	Core Reproduction Commands	34

List of Tables

5.1	Public-only corpus composition.	18
5.2	Representative model results under the shared Android adaptive-window holdout.	19
5.3	App-selectable models under the shared Android adaptive-window methodology.	22
5.4	Anomaly utility under privacy-reduced views.	23
5.5	Balanced-holdout comparison of reduced-feature deletion and hashing.	23
5.6	Strongest metadata leakage by privacy view (normalized leakage).	24
5.7	Observer leakage from encrypted-flow sequence fingerprinting.	24
5.8	Selected thesis gates and measured outcomes.	25

List of Figures

3.1	High-level MANTA data path from endpoint capture to evaluation evidence.	8
3.2	Difference between release privacy and observer leakage in MANTA.	10

Chapter 1

Introduction

1.1 Motivation

Cybersecurity monitoring is useful only when it helps protect systems against real threats. In this thesis, a threat is behavior, software, or an actor-controlled condition that can compromise a security objective such as confidentiality, integrity, or availability [24]. In mobile network traffic, relevant examples include data-exfiltration behavior, command-and-control style communication, beaconing, unusual destination changes, and traffic patterns associated with malware. An intrusion-detection system (IDS) does not observe attacker intent directly. It infers suspicious behavior from available evidence.

Mobile endpoints communicate continuously across Wi-Fi, cellular, enterprise, and home networks. Defenders therefore need visibility into traffic behavior, but that visibility is difficult to obtain without rooting the device or inspecting sensitive payload content. Android’s `VpnService` provides a practical middle ground by enabling user-space packet interception on non-rooted devices [6]. This makes endpoint telemetry possible. It also creates a design constraint: the evidence collected for detection must be useful enough for security analysis without exposing more private behavior than necessary.

Encrypted traffic makes this constraint sharper. Payload encryption removes content from inspection, but it does not remove all side channels. Flow timing, volume, burst shape, destination characteristics, and session structure can still reveal application identity or user behavior [18, 30]. Privacy is therefore not an optional concern added after detection. It constrains which telemetry is collected, stored, exported, and shared while the detector tries to identify suspicious behavior.

1.2 Terminology and Security Framing

MANTA uses the following terms throughout the thesis:

- **Threat:** behavior or software that can harm confidentiality, integrity, availability, or security accountability. MANTA focuses on network-behavior evidence of such threats, not on payload inspection or legal attribution.
- **Flow metadata:** packet-derived counters and labels such as timing, direction, byte counts, protocol, port category, app attribution, and destination/context features. This is similar in spirit to flow-monitoring systems such as NetFlow/IPFIX [9, 10].
- **Anomaly detection:** scoring behavior as unusual relative to learned or observed baselines. This is a common approach when labels are incomplete or adversarial behavior changes over time [8, 27].
- **Release privacy:** what a backend, analyst, researcher, or other recipient can infer from the feature representation that MANTA intentionally exports or shares.
- **Observer leakage:** what a passive network observer can infer from the original encrypted traffic sequence before MANTA applies hashing, bucketization, deletion, or other export-time reductions.
- **Normalized leakage:** an empirical attack-success score normalized against the baseline difficulty of the inference task. It is a measurement convenience, not a formal privacy guarantee.
- **Privacy student:** an experimental model trained to preserve anomaly-detection utility from a reduced feature view while adversary heads test whether sensitive attributes can still be inferred.
- **Federated simulation:** an experimental collaborative-learning path that trains on reduced or learned representations rather than centralizing raw full-feature tables. The prototype uses a FedProx-style objective for heterogeneous clients [1, 2, 19].

Operationally, MANTA is a pipeline. It consumes Android packet/flow metadata and public training traces. It transforms those inputs into per-app feature windows, model scores, threshold decisions, and privacy-reduced representations. It outputs local alerts, optional backend triage records, trained model files, and evaluation reports. This input-processing-output framing is important because each stage has a different security and privacy role.

1.3 Problem Statement

This thesis implements and evaluates a practical metadata-only IDS while keeping the privacy boundary explicit. The central problem is not only whether encrypted traffic

metadata can support anomaly detection. The harder problem is whether useful detection can be achieved while controlling what is revealed by the telemetry that leaves the endpoint. Payload inspection is unattractive for this setting because it is technically limited by encryption and privacy-invasive by design. Metadata-only monitoring avoids payload collection, but it is not privacy-neutral. Traffic volume, timing, direction, destination structure, and application-level patterns can still identify applications or user behavior. Reducing those features can improve privacy, but it can also remove the signal required for detection. The thesis therefore treats privacy and utility as a measured trade-off rather than as independent goals.

The project asks whether MANTA can:

1. capture and aggregate application-attributed flow metadata without payload inspection,
2. detect suspicious traffic behavior from Android-computable feature windows,
3. compare local, backend-assisted, privacy-reduced, and hybrid deployment paths,
4. quantify the utility and privacy cost of reducing exported detection telemetry, and
5. distinguish controllable release leakage from passive observer leakage visible in encrypted traffic sequences.

The same system is evaluated across detection, release-view reduction, leakage attacks, observer fingerprinting, and reproducibility packaging. This provides one framework for comparing utility, privacy, and deployment constraints instead of treating them as unrelated experiments. The thesis also makes the boundary explicit: MANTA can control the feature representation it releases, but it does not hide the original encrypted traffic pattern from an observer on the network path.

1.4 Research Questions

- RQ1: To what extent can Android-computable metadata windows support anomaly detection across a heterogeneous public encrypted-traffic corpus under grouped, source-aware evaluation?
- RQ2: Which deployment path gives the best operational trade-off among on-device, backend-assisted, privacy-reduced, and hybrid scoring?
- RQ3: How much anomaly-detection utility remains when exported telemetry is reduced through bucketization, hashing, deletion, or privacy-student representation learning?
- RQ4: What sensitive information remains inferable from released telemetry, and what information remains inferable to a passive observer outside MANTA’s feature-level controls?

1.5 Contributions

- An end-to-end implementation that connects Android metadata capture, backend triage, and a reproducible machine-learning pipeline.
- A public-only training and evaluation corpus assembled from Android malware, enterprise, IoT, campus, and mobile-app traffic datasets, with source/session-aware metadata carried through the pipeline.
- An evaluation protocol that compares local, backend-assisted, privacy-reduced, and hybrid variants under grouped source-aware holdouts instead of optimistic random splits.
- A telemetry-minimization evaluation that measures both detection utility and release leakage under reduced feature views.
- An observer-leakage benchmark that marks the boundary of feature-level privacy controls by measuring what remains inferable from encrypted-flow sequences.
- An empirical result showing that metadata coarsening can reduce exact app re-identification while still leaving substantial app-family, destination-behavior, dataset-source, and observer leakage.

1.6 Thesis Structure

Chapter 2 summarizes Android networking constraints, flow-based intrusion detection, encrypted-traffic analysis, privacy fingerprinting, and privacy-aware learning literature. Chapter 3 defines the system architecture, trust boundaries, and the distinction between release privacy and observer leakage. Chapter 4 details the implementation of the Android endpoint, backend adapter, privacy views, privacy student, and federated simulation. Chapter 5 presents the public-corpus evaluation methodology and results. Chapter 6 discusses privacy impact, ethics, and limitations. Chapter 7 closes with conclusions and future work.

Chapter 2

Background and Related Work

2.1 Android Networking and VpnService

Android exposes packet interception on non-rooted devices through `VpnService`. The service creates a user-space TUN interface and routes device traffic through an application-controlled forwarding path [6]. This design enables endpoint telemetry without kernel modification, but it also imposes practical constraints:

- a foreground service is required for long-running capture reliability [4],
- upstream sockets must be protected from tunnel recursion by calling `protect(...)`,
- packet processing must remain resource-aware to avoid user-visible overhead.

Recent Android releases also require foreground-service type declarations and, for apps targeting Android 14 or newer, corresponding Play Console declarations. VPN-style monitoring must therefore be treated as both a networking design and a platform-policy integration problem [3]. These constraints favor lightweight metadata aggregation over deep packet inspection. MANTA therefore collects packet and flow-level metadata, aggregates it on-device, and avoids payload storage.

2.2 Flow-Based Monitoring and Metadata-Only Detection

Flow telemetry abstracts packets into structured records keyed by protocol/endpoint tuples and counter/timing statistics. NetFlow v9 and IPFIX formalize this approach and define reusable fields for interoperable flow exchange [9–11, 17]. Compared with payload inspection, flow representations reduce content exposure, compress observations for efficient

storage and export, and retain behaviorally meaningful signals such as volume asymmetry, burstiness, novelty, and frequency change.

Recent anomaly-detection systems such as Kitsune and N-BaIoT show that compact flow-like representations can support intrusion detection when combined with lightweight autoencoder or ensemble methods [22, 23]. MANTA adopts this metadata-only approach for encrypted mobile and endpoint traffic. Its additional concern is deployment privacy: the same features that help detection may also reveal sensitive application or behavior information.

2.3 Encrypted Traffic Analysis and Observer Leakage

Encrypted transport does not eliminate traffic analysis. Website-fingerprinting work has repeatedly shown that timing, size, and direction patterns can reveal user activity even when payloads are fully encrypted. Deep Fingerprinting demonstrated that deep-learning attackers can defeat many lightweight defenses [18]. Later systems work shows that efficient padding schemes remain difficult to deploy robustly in realistic settings [21]. Similar risks appear in mobile traffic: partial encrypted traces can support user-activity detection [26], and concurrent-flow relationships can classify mobile applications from encrypted traffic [30].

These attacks motivate the release/observer split used in MANTA. A coarsened or anonymized export can reduce what a data consumer learns from the released feature representation. It cannot change what an on-path observer already sees in the original encrypted traffic sequence. This thesis evaluates those two leakage paths separately and treats attacker success as an empirical measurement rather than assuming that removing payloads or direct identifiers is sufficient.

2.4 Anomaly Detection for Network Security

Anomaly detection commonly operates in limited-label settings. Unsupervised or weakly supervised baselines are attractive because they can be trained on traffic that is mostly normal [8]. Isolation Forest remains a practical reference method for tabular anomaly scoring because it offers a useful performance/complexity trade-off [20]. In operational security settings, however, threshold policy is often as important as raw score quality. Base rates, alert costs, and false-positive budgets strongly affect practical utility [27].

On-device deployment adds further constraints. Inference must be lightweight, resilient under battery pressure, and explainable enough for triage. MANTA therefore combines statistical, exported local-model, and TFLite-compatible scoring with lightweight feature-contribution summaries that can be generated during inference. This follows the broader

mobile-ML trend toward local inference through TensorFlow Lite/LiteRT-style runtimes, where model format, latency, and fallback behavior are deployment constraints rather than only implementation details [14].

2.5 Privacy-Preserving Traffic Representation

Traditional anonymization techniques such as prefix-preserving pseudonymization protect explicit identifiers but may still leak semantics through traffic structure. PD-PAn argues that preserving utility while defending against semantic attacks requires attention to both prefix relationships and distributional characteristics [16]. MANTA follows that broader view. It does not only hash identifiers; it evaluates what remains inferable after identifiers and strong destination hints have been coarsened, bucketed, or removed from reduced feature views.

2.6 Privacy-Aware Representation Learning and Collaboration

Self-supervised and reconstruction-based learning for encrypted traffic has become a promising path for anomaly detection under limited labels [29]. Federated-learning research has also explored how intrusion-detection models can be improved without centralizing raw data [1, 2]. MANTA uses these ideas in a limited and measured form. A privacy student is trained from fuller supervision onto a reduced view, and a federated simulation operates on that learned latent space rather than on raw full-feature tables.

2.7 Positioning

MANTA differs from earlier work in three ways. First, it integrates Android capture, backend triage, privacy views, privacy attack benchmarks, and thesis-ready evidence generation into one evaluation framework. Second, it evaluates a public-only corpus with grouped source-aware splits rather than relying on a small controlled trace or a purely random split. Third, it treats privacy as two separate empirical questions: what the released representation leaks, and what a passive observer of encrypted traffic can still infer.

Chapter 3

System Architecture and Design

3.1 System Overview

The system consists of three modules:

- Android endpoint app (**VpnService**, flow capture, on-device scoring, UI).
- Backend adapter (authenticated ingest, queue/retry, triage, policy APIs, optional SIEM forwarding).
- ML pipeline (feature engineering, training, evaluation, threshold calibration, model export).

The architectural goal is to maximize local autonomy while still supporting optional centralized analysis and reproducible experimentation. Detection must remain possible without backend connectivity. The same feature and policy definitions must also support export, privacy-view derivation, leakage benchmarking, and model retraining.

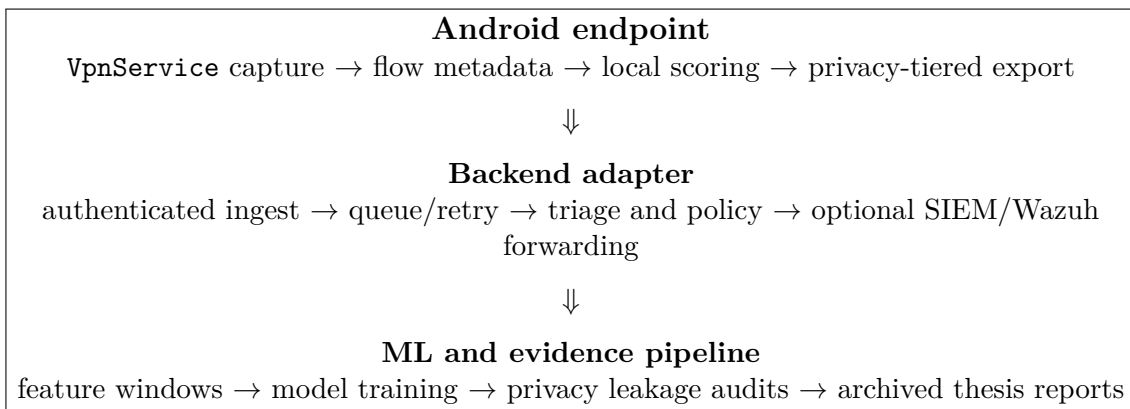


Figure 3.1: High-level MANTA data path from endpoint capture to evaluation evidence.

3.2 Data Flow

The runtime data path is:

1. Packet capture via the `VpnService` TUN interface.
2. Userspace forwarding to preserve normal network connectivity.
3. Packet parsing and app attribution to flow records.
4. Per-app rolling window aggregation. The default conceptual window is 60 s, with adaptive 30 s and 300 s modes for dense or sparse/service traffic.
5. Local anomaly scoring through statistical, multivariate, sequence, exported-tree, hybrid, TFLite-compatible, remote-assisted, and fusion modes.
6. Response-score adjustment, severity resolution, stability guards, false-positive budget checks, and alert-evidence policy.
7. Local persistence of flows, windows, and alerts in Room.
8. Optional export of selected flow and alert events to backend APIs.

Policy synchronization is decoupled through background workers. Threshold updates and retention/export changes can therefore be applied without requiring a new app build.

3.3 Threat Model and Assumptions

The primary defensive goal is to detect suspicious encrypted traffic behavior. Examples include beaconing, unusual destination changes, remote-control style communication, data-exfiltration-like volume shifts, and malware-associated anomalies. In the security framing from Chapter 1, these behaviors matter because they may indicate compromise of confidentiality, integrity, availability, or security accountability. The design assumes that payloads remain encrypted and unavailable for inspection.

The thesis considers three privacy-relevant adversaries:

- a **data consumer** who receives an exported or derived representation and attempts to infer application or behavior properties from that release;
- a **passive observer** who watches encrypted traffic on the network path and performs fingerprinting from sequence metadata alone;
- a **central coordinator** in collaborative training who may receive learned updates or reduced latent features rather than raw full-feature traffic.

The model does *not* assume root compromise of the endpoint OS, payload decryption on the wire, or strong traffic-shaping defenses such as dummy-packet padding.

Trust boundaries are defined as follows:

- **Trusted:** endpoint app process, local encrypted settings, backend adapter process.
- **Partially trusted:** network transport path, which is TLS-protected but still exposed to outages and passive observation.
- **Untrusted:** remote application peers and public network infrastructure.

3.4 Privacy Model

MANTA separates two privacy and leakage surfaces:

- **Release privacy:** what can be inferred from the feature representation that is intentionally exported, shared, stored, or used as the privacy-reduced model input.
- **Observer leakage:** what can still be inferred by a passive eavesdropper from the raw encrypted traffic sequence itself.

The distinction drives the design. Feature hashing, identifier suppression, bucketization, privacy-student training, and federated latent sharing act on the released feature representation. A passive observer sees the original encrypted sequence before those transformations occur.

Release privacy: controls and audits the feature representation intentionally exported by MANTA. This surface is affected by hashing, removal of direct identifiers, bucketization, and privacy-student learning.

Observer leakage: concerns an on-path observer who sees encrypted-flow timing, volume, direction, and sequence patterns before MANTA applies export-time reductions.

Boundary: feature coarsening can reduce release leakage, but it cannot hide the original traffic pattern from an observer on the network path.

Figure 3.2: Difference between release privacy and observer leakage in MANTA.

3.5 Telemetry-Reduction Design Decisions

The system applies data minimization and disclosure control at multiple points:

- metadata-only flow capture with no payload retention,

- hashed or removed direct identifiers in `non-off` privacy views,
- coarsened window features in the `medium` and `strict` release views,
- privacy-student training to learn a lower-leakage latent representation,
- federated-style training over privacy-student latents rather than raw full-feature tables.

These mechanisms reduce the representation that MANTA releases, but they are not formal privacy guarantees. The evaluation measures how much leakage remains after each transformation.

3.6 Deployment and Evaluation Boundaries

The design is anomaly-first rather than anonymity-first. MANTA does not implement traffic-shaping defenses such as padding, dummy traffic, or timing obfuscation. Its privacy mechanisms operate on the representation released by the endpoint or backend pipeline. The observer benchmark measures what remains visible on the network path. In other words, MANTA controls exported telemetry; it does not implement transport-level anonymity defenses.

Chapter 4

Implementation

4.1 Android Endpoint Implementation

4.1.1 VPN Capture and Userspace Forwarding

The endpoint capture loop is implemented around `FlowVpnService`. It establishes a TUN interface and starts a foreground service session. A dedicated userspace forwarder relays packets to upstream sockets while protecting those sockets from tunnel recursion through `VpnService.protect(...)` [6]. Parsed packets are aggregated into flow records keyed by app identity and network tuple attributes.

App attribution uses Android connection-owner resolution where available. Flow records are persisted to Room-backed storage [5]. Background workers handle retention cleanup and export retries through WorkManager scheduling [7].

4.1.2 Feature Windows and On-Device Detection

The feature window builder aggregates per-app behavior windows and computes a richer metadata-only feature set than the first prototype. The default conceptual window is 60 seconds. At runtime, the app selects 30-second windows for very dense traffic and 300-second windows for sparse, service, system, or background traffic. This adaptive choice reduces instability in low-volume service windows while keeping high-volume interactive traffic responsive.

In addition to core counters and ratios, the pipeline derives timing, novelty, directional balance, destination diversity, port/protocol diversity, burstiness, transport-summary features, destination-role ratios, and graph-inspired contextual signals such as destination concentration and transition rate. A second pass compares the current window with recent windows from the same app. This produces baseline-deviation features such as flow-count deviation, byte-rate deviation, novelty shift, destination-diversity shift, trend values, and

consecutive burst-window count. These features are used internally for anomaly scoring. Only fields allowed by the selected privacy tier are exported in reduced views.

On-device inference supports several runtime modes:

- **Statistical**: an online per-app multi-scale z-score baseline with warmup and feedback-aware adaptation.
- **Multivariate**: a shrinkage-covariance Mahalanobis-distance detector for correlated feature shifts.
- **Sequence**: a lightweight transition-rarity detector over bucketed behavior states.
- **Exported tree**: JSON random-forest or boosted-tree model files with fixed feature order, app-family thresholds, and optional specialists.
- **TFLite-compatible**: a TensorFlow Lite inference path for a scalar anomaly-score model deployed as `models/anomaly.tflite`.¹
- **Backend**: an optional backend inference call over a privacy-filtered feature window.
- **Fusion**: a weighted and smoothed combination of available scorers.

The TFLite path is treated as a deployable inference interface rather than as a specific algorithmic claim. The Android scorer memory-maps `models/anomaly.tflite`, builds a float tensor in `FeatureWindow.portableFeatureOrder`, and expects one scalar score in $[0, 1]$. If a reconstruction-style or one-class neural model is used, the export must expose the reconstruction error or anomaly score as that scalar output. The Android scorer does not itself compare an input vector with a reconstructed vector. If TFLite loading or inference is unavailable, the selected TFLite mode falls back to the exported local model and then to the statistical detector.

The Android implementation includes separate recall-oriented and precision-oriented tree models with user-facing names. “High-recall RF” is the recall-oriented local model. “Low-FPR RF” is the precision-oriented local model. “Adaptive hybrid RF” combines both at runtime. The hybrid mode scores both models. Malware-like and remote-access traffic keeps the larger score; service/system/other-app traffic is damped toward the quieter score; other families use a moderate blend. This remains deployable because it reuses the same metadata feature window and JSON tree evaluator rather than requiring a server model.

Alert generation is a separate stage after model scoring. The response score is adjusted by periodic-beacon evidence, concept drift, destination reputation/context, and data-quality penalties. Severity then passes through threshold profiles, app-profile adjustments, high-severity stability, false-positive budget enforcement, and an alert-evidence policy. The

¹Google now surfaces TensorFlow Lite as LiteRT in the Google AI Edge documentation; the `.tflite` file format remains supported [15].

evidence policy can store weak pending candidates and promote them only when evidence persists, when a correlated alert already exists, when known-danger destination evidence is present, or when an explicit single-window validation mode is selected. This separation prevents high-recall model windows from automatically becoming user-visible alerts while still allowing known-danger or validation traffic to surface promptly.

4.1.3 UI and Triage Workflow

The Compose UI exposes:

- consent/disclosure and capture controls,
- backend URL/token and export toggle,
- alert center with triage states (OPEN, INVESTIGATING, RESOLVED, FALSE_POSITIVE),
- local data purge and anonymized CSV snapshot export.

This supports both standalone local operation and analyst-assisted workflows.

4.2 Backend Adapter Implementation

The backend adapter is a FastAPI service with token authentication, strict request validation, and defense-in-depth response headers. Incoming events are enqueued in SQLite and forwarded with limited retry/backoff. Exhausted items move to a dead-letter queue for replay.

Beyond ingest, the adapter implements:

- alert triage APIs and incident summaries,
- per-device policy storage and auto-tuning hooks,
- policy simulation and quality summary endpoints,
- retraining sample export and forensics bundle export,
- optional Wazuh/Resistine connector routes [28].

This architecture isolates endpoint concerns from SIEM-specific integration complexity.

4.3 ML Pipeline Implementation

The ML pipeline is organized as reproducible command-line stages that operate over a merged public corpus rather than only over a controlled synthetic trace:

- public-dataset normalization and merged-corpus manifest generation,
- grouped split audits and evaluation-protocol reports,
- Android feature-contract audits and Android-style window generation,
- on-device exported-tree and TFLite-compatible export,
- hard-example mining, label-conflict auditing, source/family balancing, and threshold policy sweeps,
- episode-level alert evaluation in addition to per-window classification,
- remote backend training across multiple model families,
- privacy ablation and metadata leakage attack benchmarking,
- encrypted-flow sequence fingerprint benchmarking,
- privacy-student training and federated simulation,
- comparison-matrix and performance-gate generation.

4.3.1 Privacy Views

The privacy mechanism is implemented as explicit feature views rather than as an afterthought. `off` and `low` retain the full feature set. `medium` and `strict` replace many exact values with coarse or distribution-aware buckets. For example, activity, volume, duration, rate, burstiness, stability, and shape are represented through a small number of bucketed values. Direct host hints and several stronger destination/timing cues are removed from the exported representation.

This is not packet padding or transport obfuscation. The system does not add dummy traffic. The privacy transformation happens at the feature-representation level.

4.3.2 Privacy Student and Federated Path

The privacy-student model is trained on the reduced `medium` view with reconstruction-style pretraining and multiple adversary heads. Those adversaries try to recover app identity, app family, dataset source, context bucket, and destination behavior from the learned

latent representation. The objective is to preserve anomaly utility while reducing leakage of sensitive traffic attributes.

The federated simulator then trains over the privacy-student latent representation using a FedProx-style objective [19] rather than over raw full-feature windows. FedProx is a federated optimization method that adds a proximal term to reduce client-drift problems under heterogeneous local data. In MANTA, this is used only as a measurable collaborative-learning prototype; it does not prove secure deployment by itself.

4.3.3 Reproducibility Support

The `run_experiment_suite` path, cached privacy-view derivation, manifest reports, and comparison reports support reproducibility. The manuscript uses public-corpus back-end/privacy reruns and Android adaptive-window reports documented in the evidence bundle described in the appendix.

Chapter 5

Experimental Methodology and Evaluation

5.1 Experimental Setup

Primary quantitative results in this chapter come from the reported public-corpus experiments and the shared Android adaptive-window comparison. The evidence package includes dataset and protocol manifests, model reports, privacy ablation, metadata leakage, encrypted-flow fingerprinting, privacy-student and federated prototype reports, performance gates, and the app-selectable model comparison.

The core evaluation principle is to avoid optimistic random-row validation. Model and privacy reports use grouped splits keyed by dataset source, environment, session, app family, and time bucket wherever the report type supports them. The separate `evaluation-protocol.json` audit covers six grouped split strategies over a sampled 250,000-window subset to verify that the methodology itself is source-aware.

5.2 Datasets and Trace Collection

The corpus is a public-only merge of seven normalized datasets: Android Mischief, Android Spyware, CICAndMal2017, ITC-Net-Blend-60 scenario E, a Parrot-derived metadata slice, SDNCampus, and Westermo. The merged corpus contains 6,034,487 flows and yields 904,371 60-second windows for feature extraction. [Table 5.1](#) summarizes the dataset composition reported by `dataset-manifest.json`.

The corpus spans malware-heavy Android traffic, benign campus/application traffic, and industrial network traces. It is therefore more heterogeneous than a single-device lab trace. This makes the benchmark harder, but closer to the cross-source generalization problem that MANTA is intended to study.

Table 5.1: Public-only corpus composition.

Dataset source	Flow rows	Sessions	Label profile
Android Mischief	3,011	8	anomaly-only slice
Android Spyware	4,924	1	mixed benign/anomalous
CICAndMal2017	2,587,130	2,119	mixed benign/anomalous
ITC-Net-Blend-60 E	17,199	211	benign-only
Parrot metadata slice	373,566	1	benign-only
SDNCampus	3,000,000	30	benign-only
Westermo	48,657	3	mixed benign/anomalous

5.3 Metrics

- **Detection metrics:** precision, recall, F1, ROC-AUC, PR-AUC, and confusion matrix.
- **Privacy/utility metrics:** anomaly utility under `off`, `low`, `medium`, and `strict` views.
- **Metadata leakage metrics:** strongest-attacker accuracy and normalized leakage for app ID, app family, dataset source, context bucket, and destination behavior.
- **Observer leakage metrics:** encrypted-flow sequence fingerprinting accuracy and normalized leakage.
- **Performance metrics:** single-window and batch feature-extraction latency.

Normalized leakage is reported on a 0–1 scale after accounting for the baseline difficulty of the inference task. A value near 0 means that the attacker is near the configured baseline or chance level. A value near 1 means that the representation is highly identifying for the target attribute. The metric is an empirical privacy-risk score, not a formal privacy guarantee.

5.4 Android-Deployable Experiment Methodology

The Android-deployable experiments test how far the local Android path can improve while using only features that the endpoint can compute at runtime. They reuse the public corpus, generate Android-style adaptive windows, enforce an explicit Android feature contract, and evaluate deployable random-forest model files that can be interpreted by the Android JSON tree scorer.

The methodology added five safeguards after the early local results proved too weak:

- **Feature-contract audit:** every added feature must come from Android packet/flow metadata, destination insight, or recent local windows. Server-only and label-only fields are excluded to avoid train/deploy skew.

- **Window-construction alignment:** training windows are generated to match the runtime app-window builder more closely, including adaptive windows for sparse/service traffic.
- **Split and balance audits:** reports track source, family, and label imbalance so global metrics cannot hide a single dominant dataset or app family.
- **Label-conflict handling:** rounded near-duplicate windows with both benign and malicious labels are downweighted or excluded rather than treated as clean contradictory supervision.
- **Window and episode metrics:** per-window precision/recall/F1 are paired with episode-level alert metrics, because the Android app surfaces correlated alert episodes rather than isolated CSV rows.

The Android UI uses descriptive names for the deployable models. “High-recall RF” is the recall-oriented local model. “Low-FPR RF” is the conservative low-false-positive model. “Adaptive hybrid RF” is the runtime combination of both. These names describe operating behavior; the exact versioned files and commands are retained in the reproducibility appendix.

5.5 Anomaly-Detection Results

Table 5.2 summarizes the headline anomaly-detection variants under one shared Android adaptive-window comparison. The backend was retrained on the same Android-style adaptive feature cache and updated to consume the full portable Android feature vector rather than the older partial remote feature payload. Its threshold is calibrated on the Android validation holdout with an explicit false-positive budget instead of an earlier max-F1 or recall-heavy threshold. The calibrated boosted-tree backend reaches precision 0.898, recall 0.878, F1 0.888, PR-AUC 0.962, and ROC-AUC 0.966 on the balanced holdout, with false-positive rate reduced to 0.100.

Table 5.2: Representative model results under the shared Android adaptive-window holdout.

Model	Precision	Recall	F1	PR-AUC / ROC-AUC
High-recall RF	0.916	0.933	0.925	0.972 / 0.974
Privacy RF	0.888	0.934	0.910	0.898 / 0.948
Adaptive hybrid RF	0.952	0.845	0.895	0.977 / 0.979
Backend boosted tree, calibrated	0.898	0.878	0.888	0.962 / 0.966
Low-FPR RF	0.967	0.707	0.817	0.973 / 0.976

The results show usable signal in metadata-only windows. The strongest balanced F1 comes from the Android High-recall RF path. Adaptive hybrid RF sits between the recall-oriented and quiet local models: it recovers much of the Low-FPR RF recall while keeping a lower false-positive rate than the High-recall RF path. The backend tree model also remains strong after the feature-contract mismatch is removed. Threshold calibration eliminates the previous pathological 1.000-recall operating point. The backend is still noisier than the Low-FPR RF path, but it is no longer the worst false-positive option.

5.6 Experiment Progression and Negative Results

The experimental sequence was intentionally iterative. The first Android-local experiments used a small endpoint-safe feature set and simple deployable models. They proved that the app could score traffic locally, but their recall and F1 were too weak for an interactive security tool. The TFLite-compatible one-class path was also retained as a deployable interface experiment, but its early result had low recall. This showed that a mobile inference format is not enough. The feature contract, learning objective, and score calibration also have to match the operational detection problem.

Several later attempts were documented but not selected:

- **Larger expanded-window caches:** 500,000-window and early 250,000-window adaptive cache runs exceeded practical memory limits because the builder materialized too many derived columns across the raw flow table. The memory-safe row builder completed, but required a longer build path for the expanded experiment cache.
- **Boosted-tree export:** the deployable boosted-tree candidate completed, but did not beat the random-forest variants on the selected split. It was therefore documented as a useful comparison rather than used as the default local model.
- **HistGradientBoosting benchmark:** the non-deployable histogram gradient boosting model estimated the value of a stronger tree learner, but it was slightly worse than the selected teacher-weighted random forest and lacked an implemented Android export path.
- **Pure service specialist:** a service-only specialist improved precision but lost too much recall. The result suggested that service behavior was internally broad and label-noisy. A specialist without better labels or alert-episode logic became overly conservative.
- **Full hybrid backend and hybrid-model sweeps:** the Android runtime hybrid was implemented, but the Python exported-tree evaluator timed out while repeatedly scoring both 90-tree forests and sweeping episode policies. Full backend adaptive-window retraining with the heavier hybrid dual-channel family exceeded practical runtime in

this environment. The lighter boosted-tree backend completed on the same Android adaptive-window cache. These failures are evaluation/training-path bottlenecks rather than mobile-runtime blockers.

These negative results shaped the final interpretation. The largest gains came from fixing train/deploy mismatch, adding Android-computable temporal and destination context, auditing noisy labels, and evaluating alert episodes. Changing the classifier family alone was less important than aligning the data representation with the Android runtime and the user-visible alert policy.

5.7 Android-Deployable Model Development

The Android deployable path was iterated because early on-device models were not acceptable for an interactive app. The final reported app comparison therefore focuses on the app-selectable models in [Table 5.3](#), while this section explains the development logic qualitatively.

High-recall RF expanded the Android feature contract from the original compact set to local transport, destination-role, destination-transition, and per-app baseline-deviation features. It also added source/family balancing, hard-example mining, teacher-weighted random forests, and label-conflict downweighting. The main failure mode was not a single missing classifier family. Service windows with similar volume and diversity statistics were being treated as indistinguishable. Adding local context gave the model more axes for separating repeated service behavior from anomalous shifts.

Low-FPR RF then added short temporal memory, destination-intelligence features, a malware specialist, stricter service threshold policy, and stronger ambiguous-label handling. Its purpose was not to maximize global F1. Its purpose was to reduce the user-visible false alerts that a purely recall-oriented model could create.

Adaptive hybrid RF was added as a runtime hybrid rather than as a new trained model. It includes the recall-oriented and quiet models. Malware-like and remote-access traffic keeps the more sensitive score, while service/system/other-app traffic is damped toward the quieter score. In the shared offline comparison this yields precision 0.952, recall 0.845, F1 0.895, and FPR 0.043, showing the intended compromise between sensitivity and alert-noise control.

Episode-level analysis motivated the Android runtime policy that stores weak pending evidence and requires persistence, evidence diversity, or a high-confidence bypass before surfacing an alert. This matters because user-visible alerts are correlated episodes over time, not isolated CSV rows.

The Android runtime was also changed after the offline model iterations. Explicit local model modes can surface single-window validation candidates instead of waiting for

repeated evidence. Known-danger destination evidence can bypass the normal model threshold. The statistical detector has a cold-start risk floor for unusually suspicious first observations. These changes address a separate problem from model F1: user-visible alerts can be suppressed by stability gates even when the model score is nonzero. They are implementation safeguards and require a dedicated on-device alert study before being treated as offline detection-quality results.

5.7.1 Shared-Protocol App Model Comparison

Because the later Android models were trained and calibrated at different times, the final comparison uses one shared Android adaptive-window cache, one source-aware holdout split, and each model’s exported threshold. [Table 5.3](#) reports the balanced-holdout view. This is more informative for comparing precision and false-positive rate than the natural holdout because the natural sampled holdout is anomaly-heavy.

Table 5.3: App-selectable models under the shared Android adaptive-window methodology.

Model	Precision	Recall	F1	FPR	Threshold
High-recall RF	0.916	0.933	0.925	0.085	0.825
Privacy RF	0.888	0.934	0.910	0.117	0.995
Adaptive hybrid RF	0.952	0.845	0.895	0.043	0.790
Backend boosted tree	0.898	0.878	0.888	0.100	0.999
Low-FPR RF	0.967	0.707	0.817	0.024	0.790

This comparison clarifies the operating trade-offs. High-recall RF is the strongest reported single model by balanced F1. Privacy RF clears the target precision/recall gate under the same methodology, but it is not better than every normal model. Adaptive hybrid RF improves recall over the quiet path while keeping false positives well below the recall-oriented path. Low-FPR RF is conservative: it produces fewer false positives but misses more positive windows.

5.8 Release-Privacy Utility Trade-off

The release-privacy ablation results are shown in [Table 5.4](#). In the reported merged-corpus report, **off** and **low** retain the full Android feature contract and therefore have identical utility. **medium** and **strict** reduce exact destination and fine-grained feature information. For release leakage, these reduced views lower exact app re-identification, but they also reduce detection utility. This is the expected privacy-utility trade-off once the report is aligned with the final feature set.

Table 5.4: Anomaly utility under privacy-reduced views.

View	Precision	Recall	F1	PR-AUC / ROC-AUC
off	0.651	0.962	0.776	0.759 / 0.857
low	0.651	0.962	0.776	0.759 / 0.857
medium	0.565	0.871	0.685	0.661 / 0.756
strict	0.515	0.893	0.653	0.582 / 0.703

5.8.1 Hashing Versus Deleting Reduced Features

An additional experiment compared three privacy-reduction strategies under the same Android adaptive-window cache and the same source-aware holdout split. The goal was to distinguish semantic deletion from pseudonymization. In the *deleted* representation, the retained medium features are bucketed and every suppressed Android feature is set to zero. In the *hashed* representation, those same suppressed features are kept only as deterministic salted hash buckets. The **medium-plus** representation instead uses broad bucketization over many Android features while zeroing direct identity/threat-tag ratio fields.

Table 5.5 shows the balanced-holdout result. Hashing suppressed features recovers substantial utility compared with deletion, but it also increases semantic leakage. This supports a key privacy interpretation: hashing is pseudonymization, not removal. It can preserve stable fingerprints that help detection, but those same fingerprints can help an attacker infer app family or app identity.

Table 5.5: Balanced-holdout comparison of reduced-feature deletion and hashing.

Representation	Precision	Recall	F1	App-family acc.	App-ID acc.
Full reference	0.974	0.954	0.964	0.557	0.344
Medium-plus bucketed	0.978	0.937	0.957	0.585	0.280
Hashed suppressed features	0.971	0.868	0.917	0.590	0.239
Deleted suppressed features	0.934	0.707	0.805	0.471	0.208

5.9 Metadata Leakage from Released Views

Table 5.6 summarizes the strongest metadata leakage tasks for the main release-privacy views. The result is mixed. Exact app re-identification drops substantially from the **off** view to **medium** and **strict**. However, app-family, destination-behavior, and dataset-source leakage remain high even in the reduced views. This is why the release-privacy gate for **medium** and **strict** still fails.

In both **medium** and **strict**, exact app re-identification falls below the configured threshold. The app-family and destination-behavior checks remain false. The reduced views therefore

Table 5.6: Strongest metadata leakage by privacy view (normalized leakage).

View	App ID	App family	Dataset source	Destination behavior
off	0.282	0.722	0.927	0.999
low	0.272	0.799	0.902	0.988
medium	0.151	0.678	0.871	0.450
strict	0.135	0.603	0.646	0.580

improve release privacy for one target while leaving broader semantic leakage unresolved.

5.10 Local and Remote Models Under Privacy Views

The local/remote privacy utility report separates two questions. The first is the real Android runtime path: the local privacy model receives the normal Android feature window and applies its own internal **medium-plus** input transform before scoring. The second is a release-view path: a remote consumer receives an already reduced **medium** or **strict** export view and tries to reuse a full-feature scorer. These are not equivalent.

On a balanced 5,000-window utility sample, the app-facing local privacy runtime path reaches precision 0.904, recall 0.956, and F1 0.930. The backend scorer reaches precision 0.724, recall 0.987, and F1 0.835 on the same balanced sample, but with a much higher false-positive rate. Under already reduced **medium/strict** release views, the direct full-feature local and remote scorers still have no useful operating mode because most expected raw inputs have been removed. Reduced export views therefore require a dedicated student/reduced-view model, an internally transformed local scorer, or explicit threshold/policy calibration rather than naive reuse of the full-feature production scorer.

5.11 Observer Leakage from Encrypted Traffic

The observer benchmark evaluates what a passive on-path observer could infer from the encrypted traffic sequence. This benchmark is deliberately separate from the release views, because the observer sees the original traffic before any export-time coarsening. [Table 5.7](#) shows that observer leakage remains severe.

Table 5.7: Observer leakage from encrypted-flow sequence fingerprinting.

Task	Accuracy	Normalized leakage	Top-5 accuracy	Strongest model
App ID	0.544	0.544	0.719	MLP
App family	0.810	0.783	0.979	MLP
Dataset source	0.984	0.981	1.000	MLP
Context bucket	0.046	0.044	0.107	MLP

The reduced privacy views improve the representation that MANTA exports. They do not protect against an observer who sees timing, burst structure, and sequence-level metadata on the network path. The experiment marks the boundary between feature-level privacy controls and transport-level traffic-analysis defenses.

5.12 Privacy Student, Federated Path, and Performance

The privacy-student model improves representation privacy only partially. In the prototype **medium**-view report, adversary accuracy is reduced strongly for app ID but remains notable for app family, dataset source, and destination behavior. The federated path remains a measured collaboration prototype rather than the main detector in the evaluated configuration.

Performance gating is also incomplete. Table 5.8 summarizes the thesis-relevant gate outcomes. Single-window extraction meets the p95 target (22.5 ms) but misses the p99 target (87.9 ms versus the desired 40 ms). The feature-matrix construction itself is fast, but end-to-end single-window build time is not yet at the intended deployment threshold.

Table 5.8: Selected thesis gates and measured outcomes.

Gate	Target	Measured outcome	Result
Latency p95	≤ 40 ms	22.5 ms	Pass
Latency p99	≤ 40 ms	87.9 ms	Fail
Normalized app-ID release leakage	below threshold	medium : 0.151; strict : 0.135	Pass
Semantic release leakage	below threshold	app-family and destination leakage remain high	Fail
Observer-leakage audit	low leakage	app ID 0.544; dataset source 0.981	Fail
Detector quality	deployment-grade	F1: 0.925, 0.910, 0.895, 0.888, and 0.817 for High-recall RF, Privacy RF, Adaptive hybrid RF, Backend, and Low-FPR RF; backend FPR 0.100	Mixed

5.13 Interpretation

The evaluation supports several findings and rejects a stronger privacy interpretation.

- Metadata-only anomaly detection is technically feasible on a large public corpus.
- The calibrated backend run shows that backend inference remains a strong centralized option when remote scoring is acceptable.
- The Android-deployable comparison shows that the local path improves substantially when training windows, runtime windows, feature contracts, and label-conflict handling are aligned. The engineering conclusion is therefore “local detection is sensitive to deployability alignment”, not simply “local detection is too weak.”

- Release-privacy transformations can reduce exact app re-identification while preserving reasonable utility.
- Observer leakage remains severe without transport-level defenses.
- Runtime alert quality is not identical to per-window model quality. Episode grouping, evidence persistence, known-danger bypasses, and false-positive budgets change what the user actually sees.
- MANTA should be treated as an evaluation framework and research prototype rather than as a product-ready privacy-preserving IDS.

Chapter 6

Privacy, Ethics, and Limitations

6.1 Privacy Impact

MANTA’s privacy role follows from its threat-detection role. The detector needs telemetry to find suspicious behavior, but that telemetry can itself reveal user and application behavior. The system therefore enforces a metadata-only telemetry boundary: packet payloads are not stored or exported, and the released views progressively suppress direct identifiers and strong destination hints. From a GDPR-oriented perspective [13], this aligns with data minimization, purpose limitation, and storage-limitation principles. The design still processes behaviorally revealing metadata, so transparency, consent, and retention control remain necessary.

MANTA implements several telemetry-reduction mechanisms:

- metadata-only capture instead of payload inspection,
- hashed or removed direct identifiers in the reduced views,
- coarse or distribution-aware bucketization of many released features,
- privacy-student learning over a reduced feature view,
- federated-style learning over privacy-student latents instead of raw full-feature tables,
- explicit auditing of both release privacy and observer leakage.

These mechanisms reduce what MANTA releases or shares. They are not equivalent to a formal privacy guarantee.

The evaluation also distinguishes hashing from deletion. Hashing removes direct readability of a field, but deterministic salted hashes can still preserve stable equality and fingerprint structure. Deleting a suppressed feature removes it from the released information set entirely. The hash-versus-delete experiment in [Table 5.5](#) therefore treats hashing as

pseudonymization rather than anonymization. Hashing can recover detection utility, but it may preserve learnable app-family or app-ID signal for an attacker.

6.2 Ethical Considerations

Responsible deployment requires:

- informed consent before monitoring starts,
- understandable disclosure of what is and is not collected,
- limited retention with user-triggered purge capability,
- strict access controls on backend triage endpoints,
- constraints against repurposing telemetry for non-security surveillance.

These points are consistent with EDPB guidance on transparency and lawful processing in digital services [12]. Security hardening practices from mobile application standards, such as OWASP MASVS, further reduce secondary risk [25].

6.3 Release Privacy versus Observer Leakage

The release and observer measurements expose different failure modes. In the release view, **medium** and **strict** substantially reduce exact app re-identification. Those same views still leak high-level semantics such as app family and destination behavior. The observer benchmark then shows that a passive eavesdropper can recover strong information from encrypted-flow sequences even when the exported representation is reduced.

A release-privacy defender controls the representation that is exported or shared. A passive observer does not consume that exported representation; they consume the original traffic behavior on the network path. This is why export-time coarsening does not neutralize traffic-analysis attacks. Works such as Deep Fingerprinting and later evaluations of website-fingerprinting defenses reinforce this distinction [18, 21].

6.4 Interpretation of the Negative Result

Feature-level coarsening is not enough to provide strong privacy against passive observers. Many metadata-only monitoring designs rely on payload omission as their primary privacy argument. The MANTA results show that this argument is incomplete. The released representation can be made less revealing, but encrypted traffic remains fingerprintable at

the observer level. This is a limitation result, not a claim that MANTA solves passive traffic analysis.

6.5 Leakage and Utility Evaluation

MANTA benchmarks privacy risk with two complementary views:

- utility and leakage under feature-reduced release views, and
- observer leakage from encrypted-flow sequence fingerprinting.

The two measurements keep utility and inference risk visible at the same time. A reduced export format has to retain threat-detection signal while reducing what secondary attackers can infer.

6.6 Limitations and Threats to Validity

- The anomaly models improve substantially in the public-corpus backend run, but the result is not a full adaptive-window evaluation of every model family.
- The Android-deployable models use an adaptive-window protocol, while the backend and privacy reports use practical source-balanced caps. These results should be compared within their documented protocols rather than treated as one fully unified full-corpus run.
- Full adaptive backend training was attempted but exceeded practical memory/time on the 6.0M-flow corpus. The completed backend/privacy reports therefore use documented source-balanced raw-flow caps.
- The Adaptive hybrid RF runtime hybrid is evaluated on a bounded adaptive-window holdout, but still needs faster full-corpus scoring and a dedicated on-device alert-quality study for the final alert-evidence policy.
- The dataset mix is public and heterogeneous, but it still inherits each source dataset's collection biases and labeling noise.
- The privacy-student and federated paths are measured prototypes, not production defenses.
- Android foreground-service type declarations and Play Console review requirements create an additional deployment constraint for VPN-style monitoring on recent Android versions.

- No packet padding, dummy traffic, or timing obfuscation is implemented, so passive-observer protection remains out of scope.
- Additional privacy attacks such as membership inference, poisoning, and adaptive counterfactual adversaries remain future work.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

MANTA is an end-to-end framework for metadata-only anomaly detection over encrypted mobile and endpoint traffic. It combines Android capture, backend-assisted triage, privacy-aware feature views, privacy-student and federated variants, and a reproducible public-corpus evaluation pipeline. The main result is a boundary result: metadata coarsening can improve control over released detection telemetry, but it does not provide transport-level privacy against passive traffic-analysis attacks.

The answers to the research questions are as follows:

- **RQ1 (metadata-only detection feasibility):** Metadata-only detection is feasible on a large public corpus, but evaluation protocol matters. Under the shared Android adaptive-window comparison, High-recall RF reaches F1 0.925, Privacy RF reaches F1 0.910, Adaptive hybrid RF reaches F1 0.895, and the calibrated full-feature backend boosted-tree model reaches F1 0.888. The backend model has strong ranking quality, and threshold calibration reduces its false-positive rate from the earlier recall-heavy operating point.
- **RQ2 (variant comparison):** The deployment variants expose different operating points. High-recall RF is best by balanced F1. Privacy RF is the strongest reduced-view-compatible local path. The calibrated backend boosted tree is a centralized high-precision operating point. Low-FPR RF trades recall for fewer false positives. Adaptive hybrid RF quantifies the runtime compromise between those two local models, improving recall over Low-FPR RF while keeping much of its false-positive reduction. The privacy-oriented variants remain useful for measurement, but they are not yet competitive as privacy-only collaborative detectors.
- **RQ3 (release privacy):** The reduced privacy views lower exact app re-identification, but the reported evaluation shows a real utility trade-off. F1 falls from 0.776 in `off/low`

to 0.685 in **medium** and 0.653 in **strict**. This supports a limited release-privacy result, not a free privacy improvement.

- **RQ4 (observer leakage):** The encrypted-flow sequence benchmark shows severe observer leakage. Release privacy and passive-observer protection are therefore not interchangeable. Feature-level coarsening alone is insufficient for strong traffic-analysis privacy.

The thesis contribution is therefore not a claim that metadata-only monitoring solves privacy. The contribution is a reproducible encrypted-traffic IDS framework that separates controllable release privacy from passive-observer leakage. It demonstrates the importance of deployability-aligned Android feature engineering and shows that metadata-only coarsening can reduce exact app re-identification while remaining insufficient against stronger semantic and passive-observer inference.

The project also delivers an Android app, backend adapter, ML pipeline, report bundle, manuscript, and reproducibility appendix. Its value is strongest as an empirical systems study and evaluation framework rather than as a finished privacy-preserving product.

7.2 Future Work

- A faster full-corpus evaluator for the Adaptive hybrid RF runtime hybrid, preferably using persisted High-recall RF and Low-FPR RF score vectors rather than repeatedly interpreting both exported forests in Python.
- A dedicated on-device alert study that measures the final alert-evidence policy rather than only per-window model scores.
- Stronger remote and on-device anomaly models under the same grouped public-corpus protocol.
- Better TFLite export alignment, including scalar anomaly-score exports for reconstruction models or explicit Android-side reconstruction-error computation.
- Transport-level defenses such as padding or timing obfuscation if passive-observer protection is a hard requirement.
- Better privacy-student objectives that suppress app-family and behavior leakage more strongly while preserving anomaly utility.
- More realistic deployment studies on mobile overhead, battery, and long-running user acceptance.
- Stronger collaborative-learning paths with secure aggregation, client weighting, and robustness against poisoning.

Appendix A

Reproducibility Package

A.1 Report Bundle

The main experiment files used in this manuscript are the reported public-corpus outputs, backend and privacy evaluation outputs, and Android adaptive-window model reports. During the experiment runs these reports were written to `ml-pipeline/reports/`. The submission snapshot mirrors the selected thesis reports in the thesis export directory and its `reports/` subdirectory:

- `ml-pipeline/reports/dataset-manifest.json`
- `ml-pipeline/reports/evaluation-protocol.json`
- `ml-pipeline/reports/android-model-evaluation.json`
- `ml-pipeline/reports/tflite-autoencoder-evaluation.json`
- `ml-pipeline/reports/remote-assisted-model.json`
- `ml-pipeline/reports/remote-assisted-model-new-dataset.json`
- `ml-pipeline/reports/remote-assisted-model-parity-same-methodology.json`
- `ml-pipeline/reports/remote-assisted-threshold-calibration-same-methodology.json`
- `ml-pipeline/reports/android-backend-parity-comparison-same-methodology.json`
- `ml-pipeline/reports/android-backend-parity-comparison-same-methodology.csv`
- `ml-pipeline/reports/android-backend-parity-comparison-same-methodology.md`
- `ml-pipeline/reports/privacy-ablation.json`
- `ml-pipeline/reports/privacy-ablation-new-dataset.json`
- `ml-pipeline/reports/privacy-leakage.json`
- `ml-pipeline/reports/privacy-leakage-new-dataset.json`
- `ml-pipeline/reports/model-privacy-utility-local-remote-new-dataset.json`
- `ml-pipeline/reports/traffic-fingerprint.json`
- `ml-pipeline/reports/privacy-student-report.json`

- `ml-pipeline/reports/federated-report.json`
- `ml-pipeline/reports/performance-gates.json`
- `ml-pipeline/reports/privacy-gate.json`
- `ml-pipeline/reports/full-model-matrix.json`
- `ml-pipeline/reports/android-model-v13-expanded-report.json`
- `ml-pipeline/reports/android-model-v13-expanded-comparison.json`
- `ml-pipeline/reports/android-model-v13-expanded-error-report.json`
- `ml-pipeline/reports/android-model-v13-expanded-episode-report.json`
- `ml-pipeline/reports/android-model-v13-expanded-alert-policy-report.json`
- `ml-pipeline/reports/android-model-v13-expanded-label-audit.json`
- `ml-pipeline/reports/android-model-v13-expanded-split-balance-audit.json`
- `ml-pipeline/reports/android-model-v13-expanded-experiment-notes.md`
- `ml-pipeline/reports/android-model-v14-sequence-intel-report.json`
- `ml-pipeline/reports/android-model-v14-sequence-intel-comparison.json`
- `ml-pipeline/reports/android-model-v14-sequence-intel-error-report.json`
- `ml-pipeline/reports/android-model-v14-sequence-intel-episode-report.json`
- `ml-pipeline/reports/android-model-v14-sequence-intel-alert-policy-report.json`
- `ml-pipeline/reports/android-model-v14-sequence-intel-experiment-notes.md`
- `ml-pipeline/reports/android-model-v15-hybrid-report.json`
- `ml-pipeline/reports/android-model-v15-hybrid-experiment-notes.md`

A.2 Core Reproduction Commands

The full public-corpus workflow is documented in `ml-pipeline/README.md`. The compact reproduction path is:

```
cd ml-pipeline
python -m pip install -e .[tflite,test]

python -m ml_pipeline.dataset_manifest \
    --input data/real/manta-real-training.csv \
    --output reports/dataset-manifest.json

python -m ml_pipeline.evaluation_protocol_report \
    --input data/real/manta-real-training.csv \
    --output reports/evaluation-protocol.json
```

```
python -m ml_pipeline.train_all \  
  --input data/real/manta-real-training.csv \  
  --runs-root experiment-runs
```

```
python -m ml_pipeline.run_experiment_suite \  
  --input data/real/manta-real-training.csv \  
  --output-dir experiment-runs/real-run-001 \  
  --model-threshold 0.5 \  
  --ids-threshold 0.55
```

The later Android-deployable iterations can be reproduced with the dedicated Android training and evaluation commands recorded in the corresponding report notes. High-recall RF (v13) trains the expanded-feature deployable random-forest family. Low-FPR RF (v14) extends that path with sequence and destination-intelligence features. Adaptive hybrid RF (v15) is not a new training command; it evaluates the exported v13 and v14 model files together:

```
python -m ml_pipeline.hybrid_model_evaluation \  
  --input data/real/manta-real-training.csv \  
  --model-v13 \  
  artifacts/android/anomaly-deployable-v13-expanded.json \  
  --model-v14 \  
  artifacts/android/anomaly-deployable-v14-sequence-intel.json \  
  --output reports/android-model-v15-hybrid-report.json \  
  --window-mode adaptive \  
  --max-adaptive-windows 250000 \  
  --label-strategy window \  
  --max-fpr 0.05
```

In this package, the dedicated v15 sweep records an incomplete full-corpus evaluation because the Python exported-tree report path timed out. The Android/backend parity comparison below evaluates the same runtime hybrid on a bounded adaptive-window holdout and provides the thesis metrics. A full-corpus hybrid sweep would still require a faster offline scorer or persisted v13/v14 score vectors. The Android alert-evidence policy changes are validated by the Android implementation and unit-test suite, but they are not yet represented by a complete public-corpus alert-quality report.

The backend was then rerun against the same Android adaptive-window cache used by the app-model comparison. The heavier hybrid dual-channel backend family exceeded practical runtime in this environment, so the completed parity backend file uses the boosted-tree backend family over the full portable Android feature contract:

```
python -m ml_pipeline.train_remote_backend_model \  
  --input data/real/manta-real-training.csv \  
  --runs-root experiment-runs
```

```

--input data/real/manta-real-training.csv \
--output-model artifacts/backend/remote-assisted-model.json \
--output-report \
    reports/remote-assisted-model-parity-same-methodology.json \
--window-mode adaptive \
--max-adaptive-windows 250000 \
--label-strategy window \
--model-family gradient_boosted_tree

python -m ml_pipeline.calibrate_backend_threshold \
--input data/real/manta-real-training.csv \
--remote-model artifacts/backend/remote-assisted-model.json \
--output-model artifacts/backend/remote-assisted-model.json \
--output-report \
    reports/remote-assisted-threshold-calibration-same-methodology.json \
--window-mode adaptive \
--max-adaptive-windows 5000 \
--label-strategy window \
--max-eval-rows 5000 \
--target-precision 0.85 \
--target-recall 0.90 \
--target-max-fpr 0.18

python -m ml_pipeline.compare_android_deployable_models \
--input data/real/manta-real-training.csv \
--output-json \
    reports/android-backend-parity-comparison-same-methodology.json \
--output-csv \
    reports/android-backend-parity-comparison-same-methodology.csv \
--output-md \
    reports/android-backend-parity-comparison-same-methodology.md \
--remote-model artifacts/backend/remote-assisted-model.json \
--window-mode adaptive \
--max-adaptive-windows 5000 \
--label-strategy window \
--max-eval-rows 5000

python -m ml_pipeline.privacy_ablation \
--input data/real/manta-real-training.csv \
--output reports/privacy-ablation-new-dataset.json \

```

```
--window-mode bucket \  
--label-strategy window \  
--max-flow-rows 100000
```

```
python -m ml_pipeline.privacy_leakage_benchmark \  
--input data/real/manta-real-training.csv \  
--output reports/privacy-leakage-new-dataset.json \  
--window-mode bucket \  
--label-strategy window \  
--max-flow-rows 100000 \  
--max-class-rows 300
```

```
python -m ml_pipeline.model_privacy_utility \  
--input data/real/manta-real-training.csv \  
--local-model \  
    ../android-app/app/src/main/assets/models/anomaly-local.json \  
--remote-model artifacts/backend/remote-assisted-model.json \  
--output reports/model-privacy-utility-local-remote-new-dataset.json \  
--window-mode bucket \  
--label-strategy window \  
--max-flow-rows 25000
```

The normalized input CSV is derived from the public datasets listed in [Table 5.1](#). Raw datasets are not redistributed with the thesis archive. The archive contains the derived report bundle used for the manuscript tables.

Bibliography

- [1] Mamoun Alazab et al. Enhancing privacy-preserving intrusion detection through federated learning. *Electronics*, 2023. URL <https://www.mdpi.com/2079-9292/12/16/3382>.
- [2] Tareq H. H. Aldhyani et al. Privacy-preserving fl-based ids for cyber-physical systems. *Mathematics*, 2024. URL <https://www.mdpi.com/2227-7390/12/20/3194>.
- [3] Android Developers. Foreground service types. <https://developer.android.com/develop/background-work/services/fgs/service-types>, 2026. Accessed: 2026-07-08.
- [4] Android Developers. Foreground services overview. <https://developer.android.com/develop/background-work/services/foreground-services>, 2026. Accessed: 2026-07-08.
- [5] Android Developers. Save data in a local database using room. <https://developer.android.com/training/data-storage/room>, 2026. Accessed: 2026-07-08.
- [6] Android Developers. Vpnservice. <https://developer.android.com/reference/android/net/VpnService>, 2026. Accessed: 2026-07-08.
- [7] Android Developers. Workmanager. <https://developer.android.com/topic/libraries/architecture/workmanager>, 2026. Accessed: 2026-07-08.
- [8] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):15:1–15:58, 2009. doi: 10.1145/1541880.1541882.
- [9] B. Claise. Cisco systems netflow services export version 9. Technical Report RFC 3954, IETF, 2004.
- [10] B. Claise, B. Trammell, and P. Aitken. Specification of the ip flow information export (ipfix) protocol for the exchange of flow information. Technical Report RFC 7011, IETF, 2013.
- [11] B. Claise, B. Trammell, and P. Aitken. Information model for ip flow information export (ipfix). Technical Report RFC 7012, IETF, 2013.

- [12] European Data Protection Board. Edpb guidelines 05/2020 on consent under regulation 2016/679. https://edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-052020-consent-under-regulation-2016679_en, 2020. Accessed: 2026-07-08.
- [13] European Union. Regulation (eu) 2016/679 (general data protection regulation). <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016.
- [14] Google AI Edge. Litert: High-performance on-device machine learning framework. <https://ai.google.dev/edge/litert>, 2026. Accessed: 2026-07-08.
- [15] Google AI Edge Team. Tensorflow lite is now litert. <https://developers.googleblog.com/en/tensorflow-lite-is-now-litert/>, 2024. Accessed: 2026-07-08.
- [16] Xiaodan Gu and Kai Dong. Pd-pan: Prefix- and distribution-preserving internet of things traffic anonymization. *Electronics*, 12:4369, 2023. doi: 10.3390/electronics12204369.
- [17] IANA. Ip flow information export (ipfix) entities. <https://www.iana.org/assignments/ipfix/ipfix.xhtml>, 2026. Accessed: 2026-07-08.
- [18] Mohsen Imani, Marc Juarez, Payap Sirinam, and Matthew Wright. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018. doi: 10.1145/3243734.3243768.
- [19] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, volume 2, pages 429–450, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html.
- [20] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008. doi: 10.1109/ICDM.2008.17.
- [21] Nate Mathews, James K. Holland, Se Eun Oh, Mohammad Saidur Rahman, Nicholas Hopper, and Matthew Wright. Sok: A critical evaluation of efficient website fingerprinting defenses, 2022. Manuscript in the local thesis notes collection.
- [22] Yair Meidan, Michael Bohadana, Yisroel Mathov, Yisroel Mirsky, Daniel Breitenbacher, Asaf Shabtai, and Yuval Elovici. N-baiot: Network-based detection of iot botnet attacks using deep autoencoders. *IEEE Pervasive Computing*, 2018. URL <https://arxiv.org/abs/1805.03409>.

- [23] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. *arXiv preprint arXiv:1802.09089*, 2018. URL <https://arxiv.org/abs/1802.09089>.
- [24] National Institute of Standards and Technology. Standards for security categorization of federal information and information systems. Technical Report FIPS PUB 199, National Institute of Standards and Technology, 2004.
- [25] OWASP Foundation. Owasp mobile application security verification standard (masvs). <https://mas.owasp.org/MASVS/>, 2026. Accessed: 2026-07-08.
- [26] Madushi H. Pathmapaperuma, Yogachandran Rahulamathavan, Safak Dogan, and Ahmet Kondo. Cnn for user activity detection using encrypted in-app mobile data. *Future Internet*, 14(2):67, 2022. doi: 10.3390/fi14020067.
- [27] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316, 2010. doi: 10.1109/SP.2010.25.
- [28] Wazuh. Wazuh api reference. <https://documentation.wazuh.com/current/user-manual/api/reference.html>, 2026. Accessed: 2026-07-08.
- [29] Yuxiang Wu et al. Et-ssl: Self-supervised learning for anomaly detection in encrypted traffic. *Scientific Reports*, 2025. URL <https://www.nature.com/articles/s41598-025-08568-0>.
- [30] Guoliang Xu, Ming Xu, Yunzhi Chen, and Jiaqi Zhao. A mobile application-classifying method based on a graph attention network from encrypted network traffic. *Electronics*, 12(10):2313, 2023. doi: 10.3390/electronics12102313.